eedi-mining-misconceptions-in-mathematics

1st Place Detailed Solution

Rank	1
Team	MTH 101
Author	Raja Biswas
Topic ID	551688
Version	Original

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551688
Retrieved with Kaggle CLI on 2026-10-21.

I would like to thank Kaggle and EEDI for hosting this interesting competition! It was challenging and provided many opportunities to learn/apply new techniques. As always, I'm grateful to the Kaggle community for innovative ideas and engaging discussions. I'm excited to share my detailed solution and hope others will find it helpful!

1 Task

Given a diagnostic math MCQ, together with the correct answer and an incorrect answer, the task was to recommend top 25 misconceptions (sorted by their affinity to the incorrect answer) from a pool of 2.5k+ misconceptions. Even though the standard retrieve-rerank framework was a natural fit, it came with a few challenges:

- Current LLMs often struggle to mimic either (1) a `Novice Learner` i.e. generating the incorrect answer stemming from a specific misconception or (2) an `Expert Tutor` i.e. identifying the misconception that explains a given incorrect answer. LLMs excel at solving math problems, but they are not so great at counterfactual reasoning.
- The misconception pool ties together closely related conceptual and computational mistakes, demanding high discriminative power to spot subtle differences with precision. This is highlighted in the competition overview:

> Initial efforts to use pre-trained language models have not been successful, likely due to the complexity of the mathematical content in the questions.

- The task setup requires the model to not only do well on known misconceptions but also generalize to new misconceptions.

2 Meta Thoughts

At the start of the competition, I had the following hypotheses on what could be important for a good solution:

- Synthetic data should help with out-of-distribution generalization
- High quality synthetic data generation should be feasible since Math domain can be verified and curated objectively
- Distillation from a good teacher(s) should be very powerful, as Large LLMs are significantly better at reasoning
- Distilling Chain of Thoughts (CoT) from top LLMs (e.g. Claude 3.5 Sonnet) should help to tackle difficult examples
- Quantization method and the calibration datasets could be important. We may need to add recovery adapters to the quantized models for maintaining top 1 accuracy.
- Full training might be better than LoRA

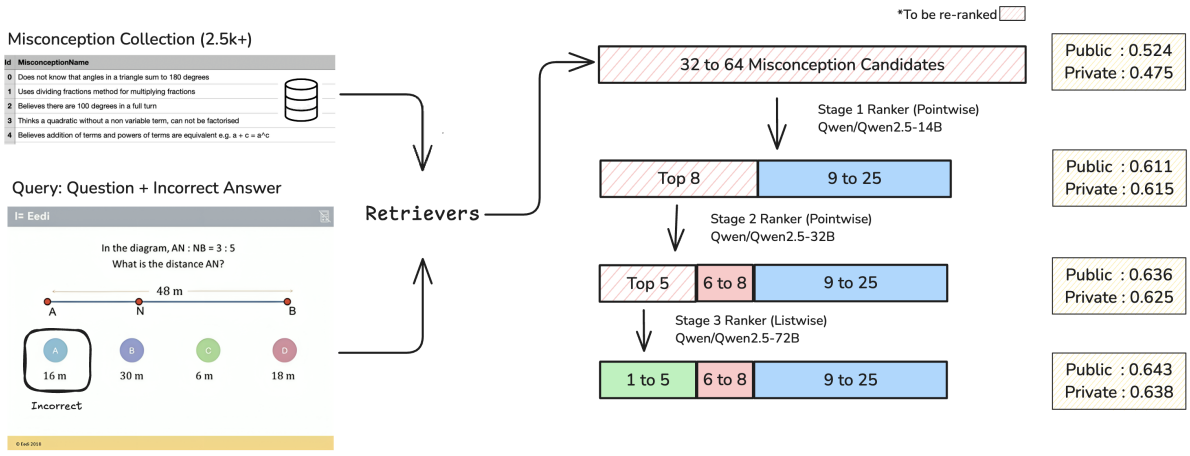
These hypotheses guided me to design my experiments throughout the competition and helped me to stick to an idea even though the initial attempts were not successful.

3 Pipeline

I implemented a retrieve-and-rerank system that involved a sequence of four steps:

- **Retrieval**: Identify top misconception candidates for each of the MCQ-Incorrect answer combinations. Specifically, I retained the top 32 retrieved candidates and included up to 32 additional candidates based on a dynamic threshold. The dynamic threshold is computed as the `top candidate similarity score - constant`. Candidates with similarity scores higher than the dynamic threshold are kept.
- **14B Ranker**: Use a fine-tuned `Qwen/Qwen2.5-14B` ranker to process all retrieved candidates and shortlist top 8.
- **32B Ranker**: Use a fine-tuned `Qwen/Qwen2.5-32B` ranker to process top 8 candidates and narrow them down to top 5.
- **72B Ranker**: Use a fine-tuned `Qwen/Qwen2.5-72B` ranker to finalize the ranking of top 5 candidates.

A schematic diagram of the pipeline and the leaderboard scores from each stages are shown below:



The final 25 misconception predictions comprised of:

- 1st to 5th candidate from the 72B ranker
- 6th to 8th candidate from the 32B ranker
- 9th to 25th candidate from the 14B ranker.

4 Data

The training data mix for both the retriever and ranker models included competition data (1.8k examples) and synthetic data (10k examples). The synthetic data played a crucial role in improving both raw performance and generalization capability with respect to unseen misconceptions.

4.1 Synthetic Data

In an ideal scenario, synthetic MCQs should have the following properties:

- The designated correct answers are indeed the correct answers (high accuracy).
- The incorrect answers directly stem from the target misconception (high diagnostic power).
- Among all misconceptions in the misconception pool (2.5k+), the indicated misconception should provide the most precise explanation for a given incorrect answer (high resolution).
- The generated MCQs should cover a broad range of subjects and constructs (high diversity).

I started generating diagnostic MCQs with a simple prompt that specified the expected format and a misconception of interest. However, it resulted in a dataset that violated almost all the properties to varying degrees. It was expected since

- The simple prompt only included information about a single misconception. Hence, the data-generating LLM was completely unaware of closely related misconceptions within the misconception pool.
- Lack of few shot examples meant lack of reference on the expected quality, difficulty level, tone, and language.
- Even the advanced LLMs struggle with handling misconceptions and counterfactual reasoning effectively.
- No curation was performed.

I experimented with several strategies to address the issues and managed to create a high quality synthetic dataset with a `grouped synthetic data generation` approach and incorporating LLM based filtering. Specifically, I first created clusters of closely related misconceptions leveraging co-occurrence statistics of misconceptions in retriever/ranker predictions on validation data. This notebook demonstrates the clustering process. Here is an example of a misconception cluster:

```
- Thinks x^2 - a^2x is a valid form of difference of two squares
- When factorising a quadratic without a non variable term, tries to double bracket factorise
- Incorrectly factorises a quadratic
- Believes the constant in an expanded quadratic comes from adding the two numbers in the brackets
- Believes the coefficent of x in an expanded quadratic comes from multiplying the two numbers in the
brackets
- Does not recognise difference of two squares
- Does not realise a quadratic must be in the form ax^2+bx+c=0 to be factorised
- Does not know how to identify common factors from algebraic terms
- Believes the difference of 2 squares method of factorising also works for the addition of 2 squares
- When factorising, believes they can choose different factors for each term
- Believes they can factorise a difference of two squares by placing the constant in both brackets
without square rooting
```

I next generated 5 new MCQs for each cluster using `Claude 3.5 Sonnet` with a refined prompt that included 4-5 reference examples on misconceptions from the cluster. I used the metaprompt tool from Anthropic cookbook for prompt engineering. Here is the final prompt used for data generation:

```
You will be generating Multiple Choice Questions (MCQs) that diagnose specific mathematical
misconceptions. Here are the misconceptions you should focus on:

<misconceptions>
{cluster_misconceptions}
</misconceptions>

Here are reference MCQs that demonstrate how to effectively diagnose these misconceptions:
```

```
<reference_mcqs>
{reference_mcqs}
</reference_mcqs>

Your task is to generate {num_mcqs} new MCQs that diagnose misconceptions not already covered by the
reference MCQs.

First, analyze the reference MCQs carefully:
1. For each reference MCQ, identify in your <analysis> tags:
   - Which misconception it targets
   - How the incorrect answers map to specific misconceptions
   - What makes the question effective at diagnosing the misconception
2. Note the style, difficulty level, and precision of language used

Then, in your <planning> tags:
- List which misconceptions still need coverage
- For each needed misconception, brainstorm mathematical contexts where it commonly appears
- Design questions where the misconception leads naturally to specific wrong answers
- Take notes on how you can craft new MCQs that adheres to the reference MCQs' style, difficulty
level, and precision of language

Finally, generate new MCQs following these important guidelines:
- Make sure each incorrect answer maps clearly to exactly one misconception
- Use precise mathematical language matching the style of reference MCQs
- Make questions challenging enough that students must demonstrate real understanding
- Ensure wrong answers are plausible and stem from genuine misconceptions, not careless errors
- Use the exact wording of misconceptions as given in the misconceptions list
- Pay careful attention to subtle differences between the misconceptions and observe which one is the
most appropriate for a given incorrect answer
- Keep the construct name and subject name as short as possible hiding the details of the
misconception
- Questions should be of higher difficulty level than reference MCQs
```

4.1.1 Curation

I used LLM-as-a-judge to filter out low quality synthetic MCQs. I prompted `GPT-4o` to rate the quality of the synthetic MCQs as below:

```
You will analyze how well an incorrect answer reflects a suspected misconception in a mathematics
problem. Your goal is to determine whether there is a clear, logical connection between the
misconception and the wrong answer.

Here is the problem with both correct and incorrect answers. The suspected misconception is also
provided:
<problem>
{PROBLEM_DATA}
</problem>

First, analyze the problem in your scratchpad:
<scratchpad>
1. Solve the problem independently to verify the correct answer
2. Examine how someone holding the suspected misconception would approach the problem
3. Trace the logical path from misconception to incorrect answer
4. Identify any gaps or inconsistencies in this connection
</scratchpad>

Then provide your evaluation using this format:
<evaluation>
1. Brief explanation of how the misconception could lead to the wrong answer
2. Score from 0-10 based on these criteria:
   - 10: Perfect alignment - wrong answer is direct result of misconception
```

```
   - 8-9: Strong alignment - clear logical path from misconception to answer
   - 5-7: Moderate alignment - connection exists but has some gaps
   - 1-4: Weak alignment - connection is unclear or requires assumptions
   - 0: No alignment - misconception does not explain wrong answer
</evaluation>

Important guidelines:
- Focus solely on the logical connection between misconception and wrong answer
- Do not speculate about other possible misconceptions
- Be specific about how the misconception leads to the error
- Flag and deduct scores if any assumptions are required to connect misconception to answer
- Consider whether a student with this misconception would consistently arrive at this wrong answer
```

I selected `GPT-4o` as judge based on vibe test as it felt more sensitive to the scoring scheme and as consistent as Claude 3.5 Sonnet.

4.1.2 Incorporating Additional Misconceptions

While creating synthetic data, LLM produced many misconceptions (4k+) that were not part of the host provided misconception pool. I decided to keep the external misconceptions to improve generalization capability of the retriever and ranker models.

Naively merging the external misconceptions with existing ones would have introduced conflicts and noise, since many of them were just rephrasing of the existing misconceptions. I adopted the following steps to handle this carefully:

- Attempt to match with existing misconceptions using string normalization (lowercasing, removing trailing punctuation, etc.)
- Attempt to match with existing misconceptions using similarity scores from an embedding model (`thenlper/gte-base`) with very high threshold (0.995)
- Remove new misconceptions that have fairly high similarity (between 0.95 and 0.995) with any of the existing misconceptions
- Keep rest of the remaining external misconceptions

4.1.3 Dataset

The final dataset is uploaded here: Eedi - Mining Misconceptions in Mathematics. It contains

- 1.8k competition MCQs + 10.6k generated MCQs
- 4791 misconceptions and follows the same format as the original competition dataset.

Reference: I'd highly recommend reading the synthetic data generation post from answer.ai: How To ~~Train~~ Synthesize Your ~~Dragon~~ Data.

4.2 Chain of Thought (CoT) Generation

To distill the reasoning capability of an advanced LLM in my solution pipeline, I first created a dataset containing thought processes generated by `Claude 3.5 Sonnet`. I provided Claude with: (i) the problem statement, (ii) the correct answer, (iii) an incorrect answer, (iv) the ground truth misconception, and (v) several closely related misconceptions. Using this information, I prompted Claude to generate the likely thought process that led a student to select the incorrect answer. The prompt is shown below:

```
You will analyze a student's incorrect answer to identify the specific reasoning flaw that led to
their error.
Your goal is to explain precisely how their misconception caused them to arrive at the wrong answer.

Here is the problem information:
<problem_data>
# Question: Simplify the expression: \[x \cdot y \cdot x\]
# Correct Answer: \(x^2y\)
# Incorrect Answer: \(x^2\)
# Primary Misconception: Ignores variables without explicit coefficients when multiplying
</problem_data>

Here are related misconceptions that are similar but do not explain this specific error as precisely:
<related_misconceptions>
- Thinks only like terms can be multiplied
- Fails to combine all instances of the same variable
- Incorrectly identifies an incomplete variable factor
- Does not understand how to multiply algebraic terms
</related_misconceptions>

First, examine all components of the problem carefully:
1. The problem statement and question asked
2. The correct answer and solution method
3. The student's incorrect answer
4. The primary misconception given
5. The related misconceptions that should be distinguished from the primary one

Then, reconstruct the student's likely thought process:
- Identify the exact point where their reasoning diverged from the correct solution path
- Note which specific mathematical operations or concepts they misapplied
- Connect their error directly to the stated primary misconception
- Verify that this explanation better fits the error than the related misconceptions

Write your analysis in <evaluation> tags, following this structure:
- Show the correct calculation first
- Show the incorrect calculations that demonstrate the error
- Explain the specific flaw in the student's reasoning
- Demonstrate how the misconception led to this particular error
- Distinguish from the related misconceptions
- Keep your explanation to 5-6 clear, non-repetitive sentences
- Focus solely on the reasoning that produced this specific error

Guidelines for writing your explanation:
- Do not restate the problem or name the misconception
- Be precise about the mathematical concepts involved
- Show exactly how the misconception led to the error
- Distinguish from related misconceptions
- Avoid repetition
- Stay focused on this specific error
```

Using this dataset, I fine-tuned three Qwen 2.5 series models (`Qwen/Qwen2.5-Math-7B`, `Qwen/Qwen2.5-14B`, and `Qwen/Qwen2.5-32B`) to act as reasoners. Notably, I omitted the true and related misconceptions from the input for fine-tuning. During inference, these models would see: (i) the problem statement, (ii) the correct answer, and (iii) an incorrect answer. They would then generate the likely thought process of a student behind selecting the incorrect answer. I used `vllm` for CoT generation during inference with `temperature=0.7, top_p=0.8, repetition_penalty=1.0, max_tokens=256`.

The CoT dataset is uploaded here: Eedi - CoT Dataset.

5 Train-Validation Split

I split the competition dataset into 5 folds, initially based on the `QuestionId` field. However, this produced overly optimistic estimates with a large gap between validation and leaderboard scores. Next, I tried GroupKFold on `SubjectId`, but this resulted in overly pessimistic estimates. I finally settled on GroupKFold with `ConstructId`, which yielded a narrow Validation/LB gap and good correlation up to ~0.62 public LB score range.

During development, I always trained my models on folds 1-4 and validated on fold 0. The synthetic data was marked as fold 99 and was included in the training set. The final models were trained on the entire dataset (full fit) using the same hyper parameters from the development stage.

6 Retrievers

I used the standard MultipleNegativesRankingLoss to fine-tune the retrievers, while referencing the FlagEmbedding codebase for developing training scripts. At this stage, I tracked both `map@25` and `recall@32` metrics and prioritized optimizing for `recall@32`, since recall was more important for overall pipeline performance.

I fine-tuned several retrievers with the following scores:

		Validation		Leaderboard	
Model	**Backbone**	**MAP@25**	**Recall@32**	**LB (Public)**	**LB (Private)**
1	intfloat/e5-mistral-7b-instruct	0.476	0.921	0.466	0.417
2	BAAI/bge-en-icl	0.493	0.909	0.463	0.419
3	Qwen/Qwen2.5-14B	0.486	**0.937**	0.449	0.451
4	Qwen/Qwen2.5-14B (re-ranker distillation)	**0.528**	0.926	**0.493**	**0.456**

Incorporating hard negatives into training batches and distilling re-ranker scores consistently improved `map@25` performance, but did not positively impact `recall@32`. For my final submission selections, I chose the high-recall `Qwen/Qwen2.5-14B` encoder (Model 3) instead of the best `map@25` encoder (Model 4). Following the same logic, I also excluded the BGE encoder (Model 2). Notably, my best encoder-only submission (Model 1 + Model 2 + Model 4) scored 0.524 on public LB and 0.475 on private LB.

Retrievers were fine-tuned using LoRA with: `r=64, alpha=128, learning_rate_lora_a=1e-5, learning_rate_lora_b=5e-5, LoRA on all linear layers, batch_size=128, and epochs=12`. A few key factors that improved recall performance were:

- Setting temperature to 0.01, instead of the typically used value of 0.02 in LLM-based encoders
- Ensuring only one demonstration per misconception appeared in each training batch, as multiple demonstrations would introduce label noise through in-batch negatives
- Pretraining the `Qwen/Qwen2.5-14B` encoder with all available synthetic data (before the curation step in section 4.1.1).

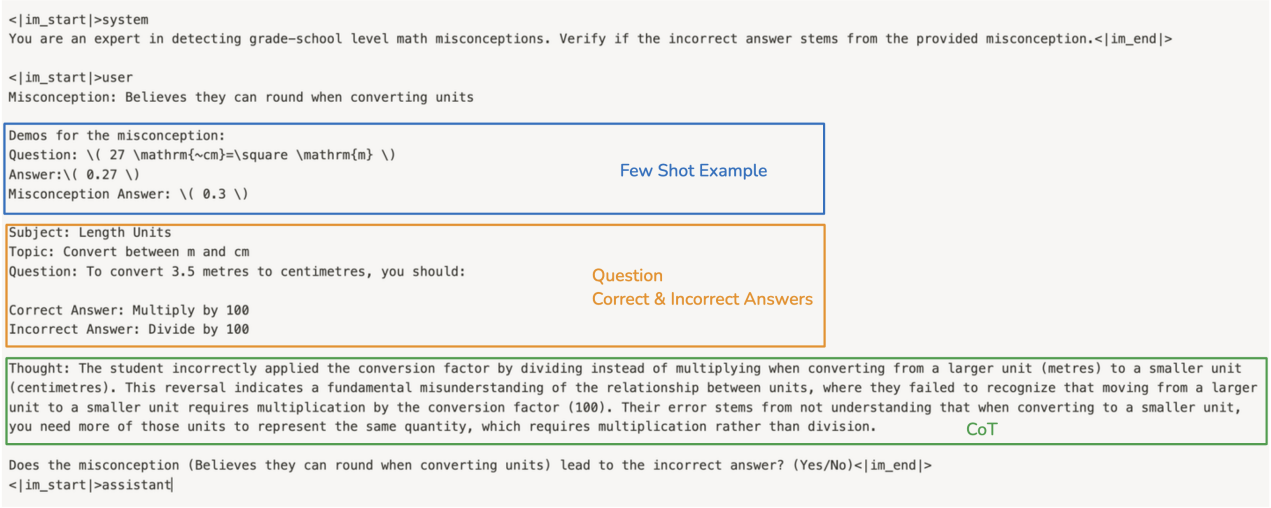
What didn't work:

- Iterative hard negative mining
- Increasing batch size through cross-device negatives

- Custom batching strategies e.g. having (query, misconception) positive pairs from the same SubjectId in the same batch
- My attempts at converting LLM retrievers to bi-directional encoders similar the the strategy used in nvidia/NV-Embed-v2.

6 Re-rankers

The Qwen 14B ranker did most of the heavy lifting by processing all retrieved misconceptions (32-64 candidates) and identifying top 8 candidates. This model was trained with a pointwise approach, where it sees one misconception at a time in the context window. The model input was structured as follows:



The logits were computed by taking the difference between the 'Yes' and 'No' token scores, as below:

```
outputs = self.model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=True)
logits_yes = outputs.logits[:, -1, self.yes_loc]  # Yes token logit at the last position [bs]
logits_no = outputs.logits[:, -1, self.no_loc]  # No token logit at the last position [bs]
logits = logits_yes - logits_no  # [bs]

logits = logits.reshape(-1, self.group_size)
labels = labels.to(logits.device).reshape(-1)
ce_loss = self.loss_fn(logits, labels)
```

Each batch contained 1 positive and `batch_size-1` negative misconceptions for a given Question-Incorrect Answer combination. The label was set to the index of the positive misconception and cross entropy loss was used for training. The models was trained with LoRA with `r=64, alpha=128, LoRA on all linear layers`. The language modelling head (`lm_head`) of the LLM (`Qwen/Qwen2.5-14B`) was frozen during fine-tuning.

6.1 Ablations

I found the following 4 strategies to be particularly significant for boosting the reranker performance and its generalization capability:

6.1.1 Few Shot Examples

To retain and leverage the LLM's in-context learning capabilities, I optionally included a few reference misconception examples (0 to 2) in the model input context (as shown in the diagram above). Not all training examples contained these few-shot demonstrations. The objective was to encourage the model to use reference examples when available, and otherwise rely on its internal reasoning. During inference, I used 1 or 2 examples from the entire training set as demonstrations. This approach was particularly effective for the 14B ranker's performance boosting private LB score from 0.495 to 0.531 **(+0.036)**. At this point, I was only using the competition data for training.

6.1.2 Distillation / Pseudo Labelling

Next, I incorporated the synthetic examples (post curation) into the training pipeline through pseudo labeling. I first fine-tuned two pointwise 72B models (`Qwen/Qwen2.5-Math-72B` and `Qwen/Qwen2.5-72B`) using only the competition dataset. These models were then used to generate pseudo labels for the synthetic examples. Finally, I used competition + pseudo-labeled data to fine-tune the next iteration of the 14B rerankers. This strategy boosted the 14B reranker's private LB score from 0.531 to 0.575 **(+0.044)**.

6.1.3 Negative Ratio

Next, I experimented with increasing the number of negative samples per positive. Showing the rankers a large number of negatives during training helped to improve performance. For each positive, I next increased the number of negatives to 24 from earlier 16. At this point, I also added scaled number of synthetic examples 2x. These two modifications boosted the 14B reranker's private LB score from 0.575 to 0.596 **(+0.021)**.

6.1.4 Chain of Thought (CoT)

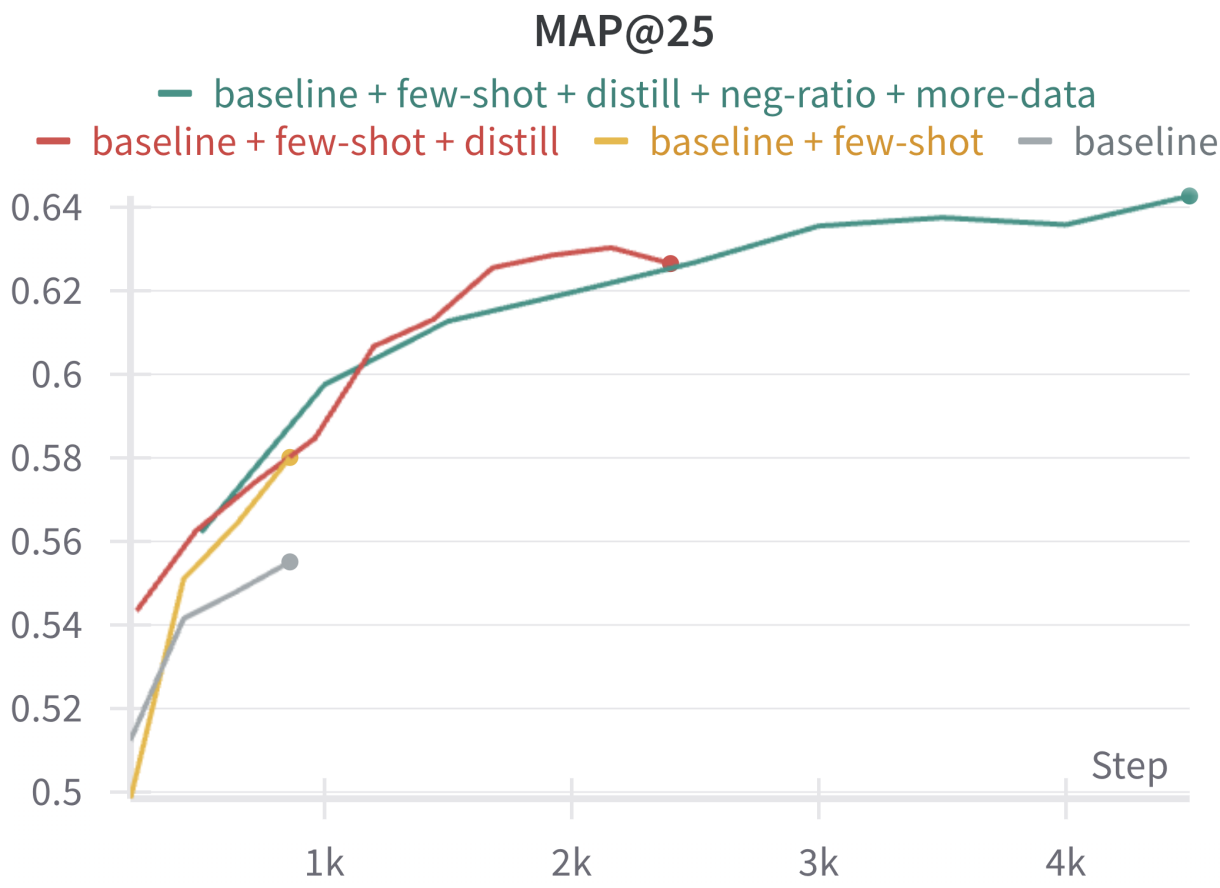
I finally added the fine-tuned CoT reasoners (Section 4.2) into training and inference pipeline. During training, CoTs produced from fine-tuned `Qwen/Qwen2.5-14B` reasoner were optionally added to the model input. 50% of training data used CoTs and 50% did not. This encouraged the model to use external reasoning when available and otherwise rely on its internal reasoning. CoT boosted the 14B reranker's private LB score from 0.596 to 0.615 **(+0.019)**.

The impact of these strategies is summarized in the following table:

Ablation (Qwen/Qwen2.5-14B)

Exp	Model	CV	LB (Public)	LB (Private)
1	Qwen/Qwen2.5-14B (Baseline)	0.555	0.545	0.495
2	Qwen/Qwen2.5-14B (Baseline + Few Shot)	0.580	0.559	0.531
3	Qwen/Qwen2.5-14B (Baseline + Few Shot + Distillation)	0.626	0.601	0.575
4	Qwen/Qwen2.5-14B (Baseline + Few Shot + Distillation + Negative Ratio + Extra Data)	0.642	0.608	0.596
5	Qwen/Qwen2.5-14B (Baseline + Few Shot + distillation + Negative Ratio + Extra Data + CoT)	**0.646**	**0.611**	**0.615**

The live eval during fine-tuning showed consistent improvement in the 14B reranker's performance.

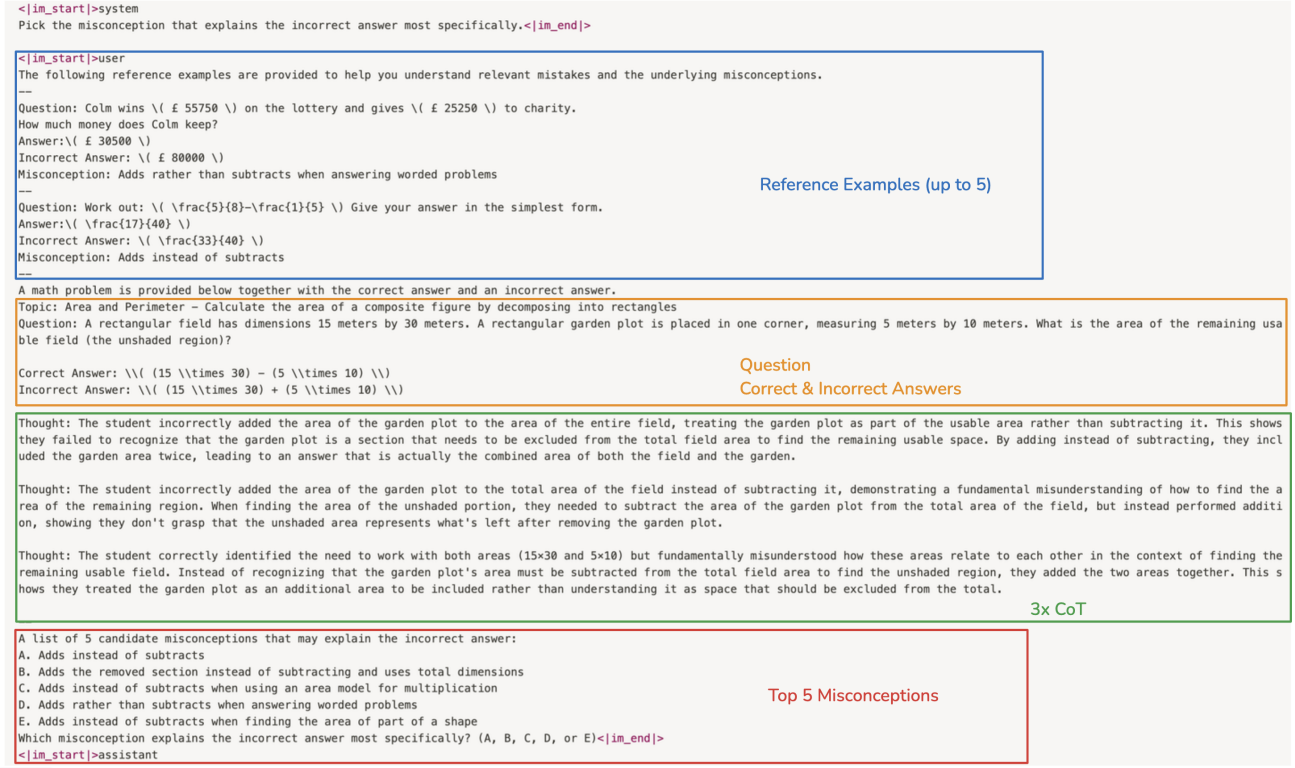


6.2 Pointwise Re-Ranker (`Qwen/Qwen2.5-32B`)

I fine-tuned the `Qwen/Qwen2.5-32B` model in exactly the same way as the 14B reranker (with a different seed). My best 32B model had validation score of `0.663` vs that of `0.646` for the 14B reranker. During inference, the 32B reranker was used to process top 8 candidates from the 14B reranker and narrow them down to top 5 candidates. Incorporating the 32B reranker boosted private LB score from `0.615` to `0.625` **(+0.010)**.

6.3 Listwise Re-Ranker (`Qwen/Qwen2.5-72B`)

The top 5 candidates were finally ranked in a listwise manner using a fine-tuned `Qwen/Qwen2.5-72B` model. Here, the model sees all of the top 5 candidates at once. An example model input is shown below:



This model had access to the even more information than pointwise re-rankers i.e.

- It sees 3 Chains of Thought (CoT) during inference - one each from the 7B, 14B, and 32B reasoners
- It sees up to 5 reference examples from the training set - 1 for each of the 5 candidate misconceptions
- It sees all of the candidates at once in the context window.

The model had the best `MAP@5` among all of the rerankers (validation MAP@5 of `0.661`). LLM based Listwise rankers typically suffer from candidate position bias. One way to mitigate this is to run multiple inference runs by shuffling the position of the candidate misconceptions (Test time augmentation (TTA) like impact). I was unable to run TTA during inference due to time constraints. Incorporating the 72B reranker boosted private LB score from `0.625` to `0.638` **(+0.013)**.

6.4 Quantization

I quantized the fine-tuned rankers using AutoAWQ. I used task specific calibration datasets for each model to mitigate performance degradation. I tested the impact of calibration dataset with one of my 32B rerankers. The results are shown below:

Model	MAP@25	Top 1 Accuracy
Qwen/Qwen2.5-32B (bf16)	0.654	0.525
Qwen/Qwen2.5-32B (AWQ - Pile as calibration)	0.649	0.518
Qwen/Qwen2.5-32B (AWQ - Task dataset as calibration)	0.652	0.526

6.5 What didn't work

- Merging strategy as described in Continuous Fine-tuning Without Loss Using Lora and Mergekit

- Qwen/QwQ-32B-Preview: I naively fine-tuned it using the same steps as with `Qwen/Qwen2.5-32B` but got worse results. The QwQ model likely requires additional research to find out its proper usage.

7 Links

- Training code
- Inference Notebook
- Dataset with Synthetic + Competition MCQ
- CoT Dataset

8 References

- Novice Learner and Expert Tutor: Evaluating Math Reasoning Abilities of Large Language Models with Misconceptions
- vllm
- How To ~~Train~~ Synthesize Your ~~Dragon~~ Data
- FlagEmbedding
- AutoAWQ

Supplemental Q&A

> **Dipam Chakraborty · 2024-12-15T11:24:21.007000**
>
> I see a lot of people used the LM Head with Yes/No probabilities as the binary classifier. Any thoughts on whether that is the best approach when fine tuning, or is it better to add a new classifier head? I tried both and the classifier head worked better for me, but it was also my first time doing any serious comparision between these two approaches. Appreciate if you share your thoughts. Thanks also for such an immensely detailed post, learnt a lot.

> **Andrey Galichin · 2024-12-15T11:36:55.657000**
>
> A very insightful and beautiful solution, thank you!
>
> We have also tried to generate Chain-of-Thought using gpt-4o in the early stage of the competition, but found that we couldn't correctly formalize the way of thinking for the model, and quite often the model explicitly mentioned the misconception in its response (although we've tried to encourage it not to do so).
>
> Haven't you met this problem during the prompt construction; if yes, can you please tell, what exact prompting modifications helped to solve it?

> **Raja Biswas · 2024-12-16T05:08:13.313000**
>
> Thanks Dipam!

I actually trained most of my models with a new classification head and switched to Yes/No tokens approach in the later stages -- mainly to take advantage of vllm inference + in-built prefix caching. Both Yes/No approach and classifier head gave similar scores here.

Raja Biswas · 2024-12-16T05:10:59.153000

Yes, I too noticed Claude mentioning the misconception name at the start of CoT. I fixed it with prompt engineering by asking LLM to follow this sequence:

- Show the correct calculation first
- Show the incorrect calculations that demonstrate the error
- Explain the specific flaw in the student's reasoning
- Demonstrate how the misconception led to this particular error
- Distinguish from the related misconceptions

xyzlsy · 2024-12-16T14:53:16.193000

Did you merge the training set and the synthetic dataset together and then perform the 5-fold split, or did you perform the 5-fold split on the training set and then directly add the synthetic data to the training set?

Raja Biswas · 2024-12-16T15:00:42.240000

5-fold split on the training set and then directly add the synthetic data to the training set

xyzlsy · 2024-12-16T15:42:50.033000

I want to know if your recall model is created by combining model 1, model 2, and model 3, and then concatenating the last token, or if another method is used.

Nikhil Tumbde · 2024-12-16T16:30:47.543000

Hi @conjuring92, thank you so much for the detailed explanation. It is quite a lot to unpack.
In my efforts towards learning, I am trying to summarize my understanding of your approach. However, I have a few questions and would really appreciate if you (or anyone on the forum) can help me get more clarity.

Questions

1. How many negative samples were used for retriever finetuning?
2. Can you elaborate on “Ensuring only one demonstration per misconception appeared in each training batch, as multiple demonstrations would introduce label noise through in-batch negatives”. Do you mean 1 misconception won’t appear as true label twice in a batch?
3. How were the negative samples obtained for retriever models 4/ 3? Using a pretrained embedding model?
4. Can you explain the re-ranker distillation part?
5. AWQ - which datasets were used for calibration?

Summary

1. Data Generation

a. Create clusters out of similar misconceptions
b. Generate 5 MCQs (using Claude 3.5 sonnet) for each clusters by giving cluster misconceptions and related 4-5 questions as context. (data quality can be questionable)
c. Use GPT-4o to evaluate the quality of generated MCQs on a scale from 0-10 and filter out the bad ones.
d. Step b gave additional misconceptions (not in misconception_mapping). Filtered out the misconceptions similar to the ones in the given mapping to avoid conflicts (using vectors from embedding model).
At this point the author has original misc clusters, and generated MCQs with misconceptions mapped to them (both original and generated)

2. COT Distillation

Overall, the idea here is to distill the knowledge of claude into smaller models. So we get generations from claude and use that to finetune smaller models.
a. Generate COT (on original train dataset) using Claude using the question context, misc and related misc in the prompt.
b. Finetune 3 models (7,14,32B) on the generated text and same prompt but exclude the misconceptions and related misc (as during inference they won't be present)
At this point the author has 3 models (reasoners) with the capability of generating COT reasons for each incorrect answer using only the questions context (no misconception).

3. Models - retriever

a. Finetune retrievers using MNR loss for 12 epochs on entire data (except 1 fold) from step 1.d. {some confusion here will update once it is clarified}
b. Pre-training the finetuned qwen-14B on synthetic data from step 1.b gave recall boost.

4. Model –reranker

overall 2 pointwise rerankers (yes/no output for each misconception) and 1 listwise used.

a. Pointwise rerankers (14/32B)

i. Pseudo labels on synthetic dataset (from 1.d) generated using models (Qwen/Qwen2.5-Math-72B and Qwen/Qwen2.5-72B) finetuned on train dataset.
ii. Entire dataset used for finetuning pointwise reranker. COT was also added to 50% of the

training dataset.
iii. Other details-

1. The prompt contains few shot examples.
2. The re-ranker was trained on logit difference of 'yes' and 'no' tokens as predictions using cross entropy loss.
3. 1 positive and (BS – 1) negative samples taken for each batch. 1:24 positive:negative ratio.
4. LM head was frozen during finetuning.

b. Listwise reranker

i. Predicts 1 out of top 5 choices.
ii. {Training strategy not shared}

5. Quantization

a. Fp16 finetuning was performed and the models were quantized using AWQ.
b. Calibration dataset used {some confusion here will update once it is clarified.}

Raja Biswas · 2024-12-17T16:22:09.333000

Thanks Nikhil for the questions. Please find below my replies.

> How many negative samples were used for retriever finetuning?

I used the standard **in-batch negatives** setup. Therefore, for a batch_size of 128, there were 127 negatives and 1 positive for each query (question + incorrect answer).

> Can you elaborate on "Ensuring only one demonstration per misconception appeared in each training batch, as multiple demonstrations would introduce label noise through in-batch negatives". Do you mean 1 misconception won't appear as true label twice in a batch?

Say, In a batch, the positive examples are `(q1, m1), (q2, m2)..., (q128, m128)`. For the in-batch negatives setup, this means `q1` will have `m1` as positive and `[m2, m3, ..m128]` as negatives. Similarly, for `q2` you will have `m2` as positive and `[m1, m3, ...m128]` as negatives. During training, we will compute similarity scores between all query and misconception combinations (`128x128` matrix). The diagonal elements would correspond to the positives and off-diagonals should be negatives. In this competition, we had many-to-one relationship for the query to content mapping. Hence, a random batching strategy would lead to cases where multiple queries belonging to the same misconception would end up in a given batch. In this case, you will have false negatives in the off diagonals -- leading to noisy, sub optimal convergence.

> How were the negative samples obtained for retriever models 4/ 3? Using a pretrained embedding model?

I just used in-batch negatives. I, however, experimented mined hard negatives obtained from fine-tuned retrievers (+ de-noised with re-ranker predictions Reference), but it was ineffective to improve long tail recall.

> Can you explain the re-ranker distillation part?

You can refer to the distillation loss in the FlagEmbedding repo.

> AWQ - which datasets were used for calibration?

I basically used the a subset of re-ranker training dataset as calibration dataset.

Raja Biswas · 2024-12-17T16:22:45.953000

I averaged similarity scores from these models.

Yannan Chen · 2024-12-21T05:39:43.933000

Thank you so much for this detailed solution. I'm so impressed with how much attention you've paid to details, and how you fully utilze the limited GPU resources to perform so many rounds of reranking. May I ask:

- you mentioned "add recovery adapters" to quantization, what is it? did you apply it eventually?
- What are the important tricks to fit 72b model for inference into 2 T4 GPUs?
- What total running time did you manange to achieve during competition? Any out of time or out of memory experienced near the end of the competition?

Thanks again! Congratulations on 1st place! The solution really deserves it.

Raja Biswas · 2024-12-21T11:19:04.580000

Thanks for the kind words! Congratulations to you too on solo gold!

> you mentioned "add recovery adapters" to quantization, what is it? did you apply it eventually?

The idea comes from the Apple Intelligence report (Section 5.2).

> … The quantized model often shows a moderate level of quality loss. Therefore, instead of directly passing the quantized model to application teams for feature development, we attach a set of parameter-efficient LoRA adapters for quality recovery. We make sure that these LoRA adapters training recipes are consistent with pre-training and post-training processes.

I didn't get to try this in this competition.

> What are the important tricks to fit 72b model for inference into 2 T4 GPUs?

I mainly followed this great post from @jagatkiran! Didn't have any inference tricks for the 72b model. For 14b ranker, prefix caching was key.

> What total running time did you manange to achieve during competition? Any out of time or out of memory experienced near the end of the competition?

My best selected submission was ~510mins. I didn't face OOM but had a few out of time near the end…

Thanks!

Geremie Yeo · 2024-12-21T12:56:21.633000

Hi Raja, thanks for the detailed solution! Congrats for the solo win

Would you mind if I ask a few questions ?

Great job on trying out lots of experiments within a short period of three months. What GPU specifications/clusters did you use to fine tune the models ?

Context: I rented H100s from RunPod and finetuning using LoRA for a 14B Yes/No reranker took > 5 hours with our team's external data. For initial experiments I tested various numbers of hard negatives (1,2,4,8,16,24) and easily training these six models can take more than 2 days total (just for one idea!). Towards the end I finetuned 32B and the training on just 4 negatives took > 20 hours for a single experiment (5 hours without external data)! I trained for 2 epochs using the SFTTrainer. Felt that there may be a faster way…

Do you have tips on selecting experiments?

Context: From the MCTS and EEDI competitions, I realised that I'm able to think of lots of ideas to solve the problem. However, I felt my ability to decide which one to prioritise needs improvement. For example in EEDI, I spent 3 days customizing TextGrad to optimize prompts for Qwen to hopefully improve the reranking performance (similar to this). However the improvement is very marginal and possibly not worth 3 full days of effort…Maybe I will get better at this as I gain more experience 😅

Really inspiring that you got the correct approach from the beginning and have 3 months to get the most out of it 👍

Raja Biswas · 2024-12-21T21:54:46.657000

Hi bogoconic1!

Thanks a lot! You've been on an impressive path and performing well across different types of competitions!

> Great job on trying out lots of experiments within a short period of three months. What GPU specifications/clusters did you use to fine tune the models ?

I used 2xH100 GPUs from vast.ai. I trained my models using the HF accelerate with a few basic speed optimizations e.g. using (1) torch.compile (2) flash-attention (3) LoRA instead of QLoRA (4) padding sequence lengths to multiples of 8. The 14B re-ranker was also taking ~5hrs for a full run in my setup. But, I very rarely did a full run: (a) ~50% of training was giving good indications on the experiment, (b) I primarily experimented with the 7B models. For 32B, I just copied the best settings from 14B. IMO key is to keep experiment setup as lean as possible.

> Do you have tips on selecting experiments?

I often struggle with this too. It's difficult to get the exploration vs exploitation balance right. Although there is no silver bullet, I think the following points help in NLP comps:

1. Try to find the best set of backbones to fine-tune from
2. Once you find a good backbone & have a decent baseline, focus on data related experiments
3. Write down takeaways after each experiment -- include related plots, error analysis etc. This should inform you what to do next
4. Techniques like distillation/pseudo labeling almost always help with performance and generalization

> Will add to this list if I can think of other points tomorrow. I'm not sure if these are too obvious or very opinionated…

> **Yannan Chen · 2024-12-22T00:36:24.273000**
>
> Thank you for your explanation! I also noticed that for your retriever, you used different learning rate for lora_a and lora _b, this is an interesting detail. May I ask what is the rationale behind that?

> **Atharva Ingle · 2024-12-22T12:32:24.967000**
>
> Thanks for the detailed writeup, a lot to learn here. I am a bit confused in this part:
>
> Question: Since you are finetuning the causal lm to output Yes/No token, why the lm_head was frozen. What was the reasoning behind this ? As we want the lm_head to have high probs for either Yes/No token, freezing it will avoid it to learn this mapping. Can you explain this part ?
>
> > The logits were computed by taking the difference between the 'Yes' and 'No' token scores, as below:
>
> ```
> outputs = .model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=)
> logits_yes = outputs.logits[:, -, .yes_loc]
> logits_no = outputs.logits[:, -, .no_loc]
> logits = logits_yes - logits_no
> logits = logits.reshape(-, .group_size)
> labels = labels.to(logits.device).reshape(-)
> ce_loss = .loss_fn(logits, labels)
> ```
>
> > Each batch contained 1 positive and batch_size-1 negative misconceptions for a given Question-Incorrect Answer combination. The label was set to the index of the positive misconception and cross entropy loss was used for training. The models was trained with LoRA with r=64, alpha=128, LoRA on all linear layers. The language modelling head (lm_head) of the LLM (Qwen/Qwen2.5-14B) was frozen during fine-tuning.

> **YuhaoWu123 · 2024-12-22T13:49:41.320000**
>
> Thank you very much for your insights! This is a very interesting and solid solution. However, I have a few questions regarding the prompt design that I'd like to clarify:
>
> 1. You mentioned that the misconception you provided are very similar, but for a single question (Q), the three incorrect answers (A) might correspond to very different types of misconception. In that case, does each reference correspond to one QA pair, or does one Q correspond to three different A options ?
> 2. The prompt does not explicitly mention that the types of misconception in the generated samples can be new. However, it does state that you need to first identify which misconception need to be leverage, which implies selecting misconception from those provided that were not covered by the reference questions. If this is the case, how can we ensure that new types of misconception will appear in the constructed dataset?
> 3. The prompt mentions that the tag needs to indicate which misconception each incorrect answer corresponds to. Does this mean that the reference questions only provide the incorrect answers without providing the misconception ?

4. Many of my above questions stem from uncertainty about the content within the tag and the tag in a complete prompt. Would you be able to provide me with an example of a fully constructed prompt including these tags?Thanks a lot again ! ~~~

Miles · 2024-12-23T00:42:09.683000

Congrats for the winning! Did you try to train the 14b reranker with synthetic data but without distillation from 72b? I wonder how it would perform.

Raja Biswas · 2024-12-23T04:48:38.020000

I followed the LoRA+ paper: LoRA+: Efficient Low Rank Adaptation of Large Models -- which shows faster convergence and better perf with different LRs for LoRA A and B.

> …Low Rank Adaptation (LoRA) as originally introduced in Hu et al. (2021) leads to suboptimal finetuning of models with large width (embedding dimension). This is due to the fact that adapter matrices A and B in LoRA are updated with the same learning rate. Using scaling arguments for large width networks, we demonstrate that using the same learning rate for A and B does not allow efficient feature learning. We then show that this suboptimality of LoRA can be corrected simply by setting different learning rates for the LoRA adapter matrices A and B with a well-chosen ratio. We call this proposed algorithm LoRA+. In our extensive experiments, LoRA+ improves performance (1-2 % improvements) and finetuning speed (up to ~ 2X SpeedUp), at the same computational cost as LoRA.

Raja Biswas · 2024-12-23T04:54:24.810000

Interesting, this should be a good experiment! However, I didn't try this thinking synthetic data misconceptions still may be noisy (lesser quality than competition data) even after curation…

Raja Biswas · 2024-12-23T05:59:35.763000

Hey @atharvaingle, on freezing the `lm_head`:

- `logit(y=Yes) = features . W_lm_head_yes` : The Yes/No token logits are controlled by both the LM head weights at the location of Yes/No tokens and model hidden states at end of the sequence. Even if we freeze the LM head, the logits can be adjusted by the network by updating other params such that features are moving as per loss during fine-tuning.
- The prompt template would already increase logits for the Yes/No tokens (without fine-tuning the LM head)
- Qwen2.5 tokenizer has 150k+ tokens (out of which we are only concerned with 2 tokens), unfreezing LM Head would increase VRAM requirement significantly
- However, as always, we should test both unfreeze and freeze cases and see the performance diff. Intuitively, I felt difference wouldn't be significant.

Raja Biswas · 2024-12-23T06:06:25.877000

Here is an example prompt:

```
You will be generating Multiple Choice Questions (MCQs)  diagnose specific mathematical
misconceptions. Here are the misconceptions you should focus on:

<misconceptions>
```

```
- Thinks  the perimeter  a square grid  the number  squares surrounding the rectangle
- Calculates area  asked  perimeter
- Counts a side length more than once  counting perimeter due   marking a starting point
- Thinks  the perimeter  a square grid  the number  squares inside  are touching the perimeter
the shape
- Calculates perimeter  asked  area
- When calculating perimeter, only adds two sides
- Believes lines inside the shape are included  finding perimeter
- When calculating perimeter, only adds the marked lengths
- When calculating perimeter, multiplies instead  adds
- Thinks only labelled dimensions need  be included  calculating the perimeter
- Confuses perimeter  area
- Fails  use the properties  simple shapes  work out missing lengths
- Does  know  we can use the conservation  perimeter  find missing lengths
- Makes an assumption about line segments being equal within a shape
- Mixes up width  height  a rectangle
- Does  know how  find missing lengths  a composite shape
</misconceptions>

Here are reference MCQs  demonstrate how  effectively diagnose these misconceptions:

<reference_mcqs>
[{: , : , : , : , : , : , : , : , : , : , : , : },
{: , : , : , : , : , : , : , : , : , : , : , : },
{: , : , : , : , : , : , : , : , : , : , : , : },
{: , : , : , : , : , : , : , : , : , : , : , : },
{: , : , : , : , : , : , : , : , : , : , : , : }]
</reference_mcqs>

Your task   generate   MCQs  diagnose misconceptions  already covered  the reference MCQs.

First, analyze the reference MCQs carefully:
 For each reference MCQ, identify  your <analysis> tags:
   - Which misconception  targets
   - How the incorrect answers   specific misconceptions
   - What makes the question effective at diagnosing the misconception
 Note the style, difficulty level,  precision  language used

Then,  your <planning> tags:
- List which misconceptions still need coverage
- For each needed misconception, brainstorm mathematical contexts where  commonly appears
- Design questions where the misconception leads naturally  specific wrong answers
- Take notes  how you can craft  MCQs  adheres  the reference MCQs
```

Raja Biswas · 2024-12-23T06:17:30.400000

> You mentioned that the misconception you provided are very similar, but for a single question (Q), the three incorrect answers (A) might correspond to very different types of misconception. In that case, does each reference correspond to one QA pair, or does one Q correspond to three different A options ?

For each MCQ, the LLM would generate

therefore, incorrect answers and misconceptions can be paired properly.

> The prompt does not explicitly mention that the types of misconception in the generated samples can be new. However, it does state that you need to first identify which misconception need to be leverage, which implies selecting misconception from those provided that were not covered by the reference questions. If this is the case, how can we ensure that new types of misconception will appear in the constructed dataset?

Nice spot. Even if we specify the possible misconceptions, LLM still likes to generate additional ones. Before I settled on this prompt, my earlier prompts would generate even more of new misconceptions.

> The prompt mentions that the tag needs to indicate which misconception each incorrect answer corresponds to. Does this mean that the reference questions only provide the incorrect answers without providing the misconception ?

I have provided a full prompt example here. Hope it helps!

YuhaoWu123 · 2025-02-12T05:06:11.393000

Hello, I'd like to ask another question. The problem ultimately uses the MAP@K of the correct answers, but your code calculates the MAP@K for all the top K answers. I'm curious why the metric in the code is inconsistent with the metric required by the problem. the code i mentioned is
for i, p in enumerate(predicted):
first condition checks whether it is valid prediction
second condition checks if prediction is not repeated
if p in actual and p not in predicted[:i]:
num_hits += 1.0
score += num_hits / (i + 1.0)

```
/ (len(actual), k)
```

looking forward for your reply，thanks a lot！

YuhaoWu123 · 2025-02-12T16:21:11.237000

hi～ I would like to ask you another question. In your notebook at https://gist.github.com/rbiswasfc/8d2bfec5c2a358e8beeb2df390111f9d, you used a file named ranker_oof_df.csv. In your solution, you mentioned, "Specifically, I first created clusters of closely related misconceptions leveraging co-occurrence statistics of misconceptions in retriever/ranker predictions on validation data." My question is, when you refer to "validation data" here, do you mean the predictions on the validation data obtained by training a model using the training set, which is split from the original entire dataset into training and valid sets? thanks a lot！

Raja Biswas · 2025-02-13T11:41:31.767000

Hi @yuhaowu123, yes, exactly. Using top retrieved candidates on the validation split.

Raja Biswas · 2025-02-13T11:42:27.030000

> Metric computation looks okay to me. Here is a detailed post on the metric: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/534077